Documento de arquitectura — Matera inteligente

UNIVERSIDAD SERGIO ARBOLEDA — ARQUITECTURA DE SOFTWARE — PROYECTO DE CORTE 2
Roger Bohórquez · Juan Joya · Andrés Céspedes
github.com/Pipelol0723/matera-inteligente · commit fff4ebb
15 de octubre de 2026

El sistema es el núcleo de la matera inteligente. El corte 1 entregó el diagnóstico: una API que recibe una especie y una medición de humedad del sustrato, luz y temperatura, y devuelve el estado de la planta. El corte 2 agrega cuentas de usuario, administración, las plantas propias de cada usuario, sensores físicos vinculados a ellas y su historial de mediciones, sin modificar el diagnóstico original. El backend (Python 3.13 y Flask) mantiene sus cuatro capas: presentación, aplicación, dominio e infraestructura. La tabla de especies y todo lo nuevo del corte 2 viven ahora en SQLite, consultadas por los mismos puertos declarados en el dominio. El front sigue siendo un cliente estático servido aparte, que usa fetch y guarda el token de sesión en sessionStorage. La API expone 19 endpoints bajo /api/v1 (antes 3) y todo error responde JSON con la forma {error, mensaje, detalle}. Hay 167 pruebas: 40 del dominio, que corren sin servidor ni base de datos, 43 de aplicación y 84 de integración. Las referencias archivo:línea son relativas a backend/ salvo que digan otra cosa.

1. Diagrama de paquetes

Cada caja es una carpeta real del repositorio. Las flechas entre capas no cambiaron respecto al corte 1: ninguna sale de dominio/, que sigue usando solo la biblioteca estándar de Python (tests/dominio/test_arquitectura.py:34 lo verifica). Lo que cambió es el contenido de cada carpeta: aplicación ganó cinco módulos de casos de uso (cuentas, administracion, plantas, sensores, historial), presentación ganó cuatro blueprints nuevos y un módulo de autenticación, e infraestructura reemplazó el CSV por un paquete sqlite/ con seis repositorios, más seguridad.py y reloj.py.

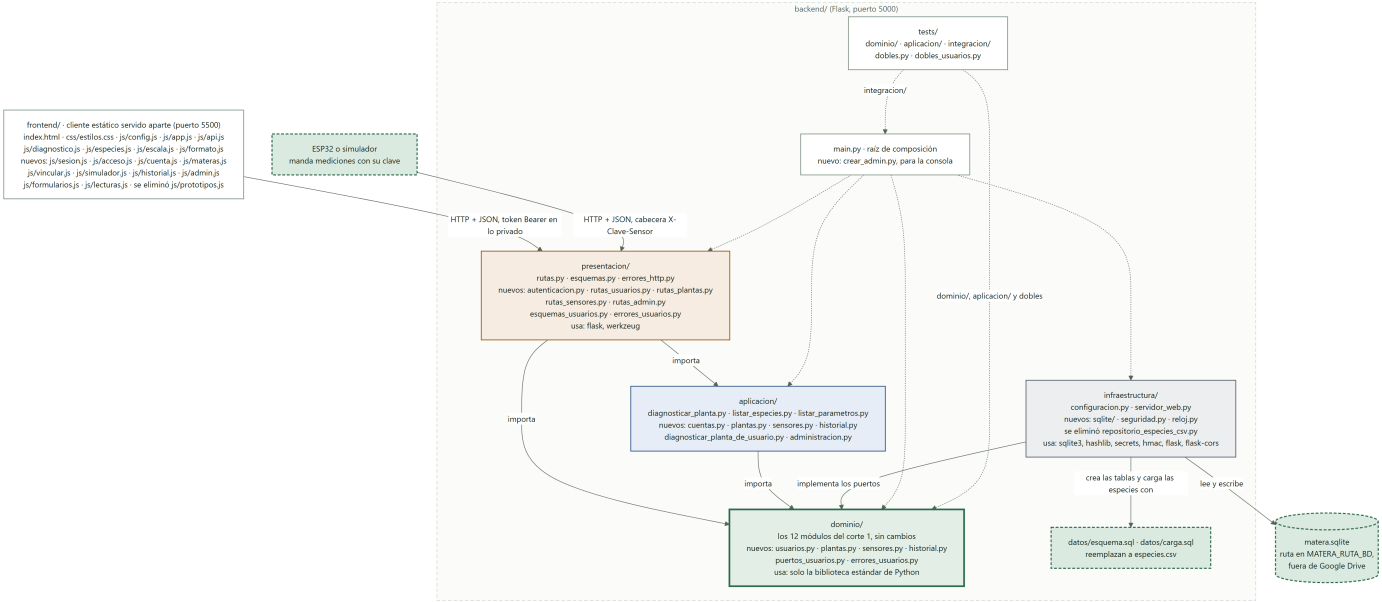


Figura 1. Paquetes del corte 2 (fuente: docs/diagramas/paquetes-corte2.mmd).

El código nuevo en infraestructura/seguridad.py no sabe nada de usuarios ni de sesiones: solo implementa los puertos Cifrador y GeneradorSecretos con hashlib, hmac y secrets de la biblioteca estándar. BaseDatos (infraestructura/sqlite/base_datos.py) es la única clase que abre conexiones SQLite; cada repositorio la recibe por constructor y nunca construye su propia conexión.

2. La API completa

Método y ruta	Qué hace	Requiere sesión
GET /api/v1/especies	Lista el catálogo de especies (público, corte 1)	No
GET /api/v1/parametros	Lista los parámetros medibles y sus límites físicos	No
POST /api/v1/diagnosticos	Diagnostica una medición suelta (corte 1, sin guardar nada)	No
POST /api/v1/usuarios	Registra una cuenta nueva (rol USUARIO siempre)	No
POST /api/v1/sesiones	Inicia sesión: devuelve el token una sola vez	No
DELETE /api/v1/sesiones/actual	Cierra la sesión actual	Sí
GET /api/v1/usuarios/yo	Mi cuenta: datos propios y cantidad de plantas	Sí
GET /api/v1/plantas	Mis materas: plantas, sensor y última medición de cada una	Sí
POST /api/v1/plantas	Registra una planta propia (apodo y especie)	Sí
DELETE /api/v1/plantas/<id>	Elimina una planta propia, con su sensor e historial	Sí
POST /api/v1/plantas/<id>/sensor	Vincula un sensor a una planta propia	Sí
DELETE /api/v1/plantas/<id>/sensor	Desvincula el sensor de una planta propia	Sí
GET /api/v1/plantas/<id>/historial	Historial de mediciones de una planta propia	Sí
POST /api/v1/plantas/<id>/mediciones	Medición manual de una planta propia	Sí
POST /api/v1/sensores/<codigo>/mediciones	Medición de un sensor físico (clave propia, no sesión)	Clave de sensor
GET /api/v1/admin/usuarios	Lista todas las cuentas con su cantidad de plantas	Admin
PATCH /api/v1/admin/usuarios/<id>	Activa/desactiva o cambia el rol de una cuenta	Admin
DELETE /api/v1/admin/usuarios/<id>	Elimina una cuenta, con todo lo suyo	Admin
POST /api/v1/admin/especies	Agrega una especie nueva al catálogo	Admin

No existe ninguna ruta pública para crear un administrador. El primer admin se da desde la consola del servidor, fuera de la API: python crear_admin.py correo (backend/crear_admin.py), que llama a ConvertirEnAdministrador (aplicacion/administracion.py) sin pasar por HTTP.

3. Diagramas de secuencia

El del diagnóstico (corte 1) no cambió en su lógica: solo cambió quién implementa ConsultaRangos (SQLite en vez de CSV). Se agregan tres secuencias nuevas del corte 2.

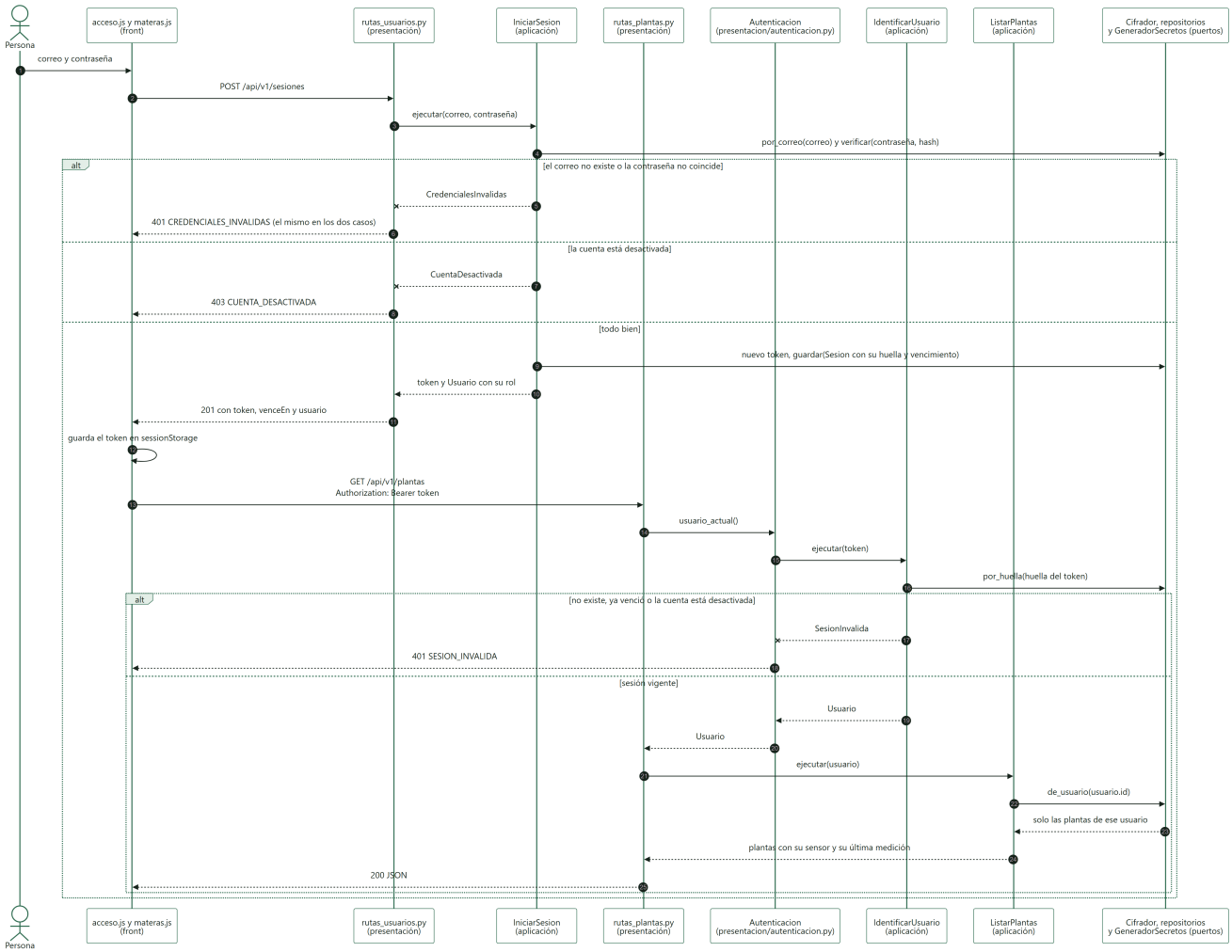


Figura 2. Login y «mis plantas» (fuente: secuencia-login.mmd).

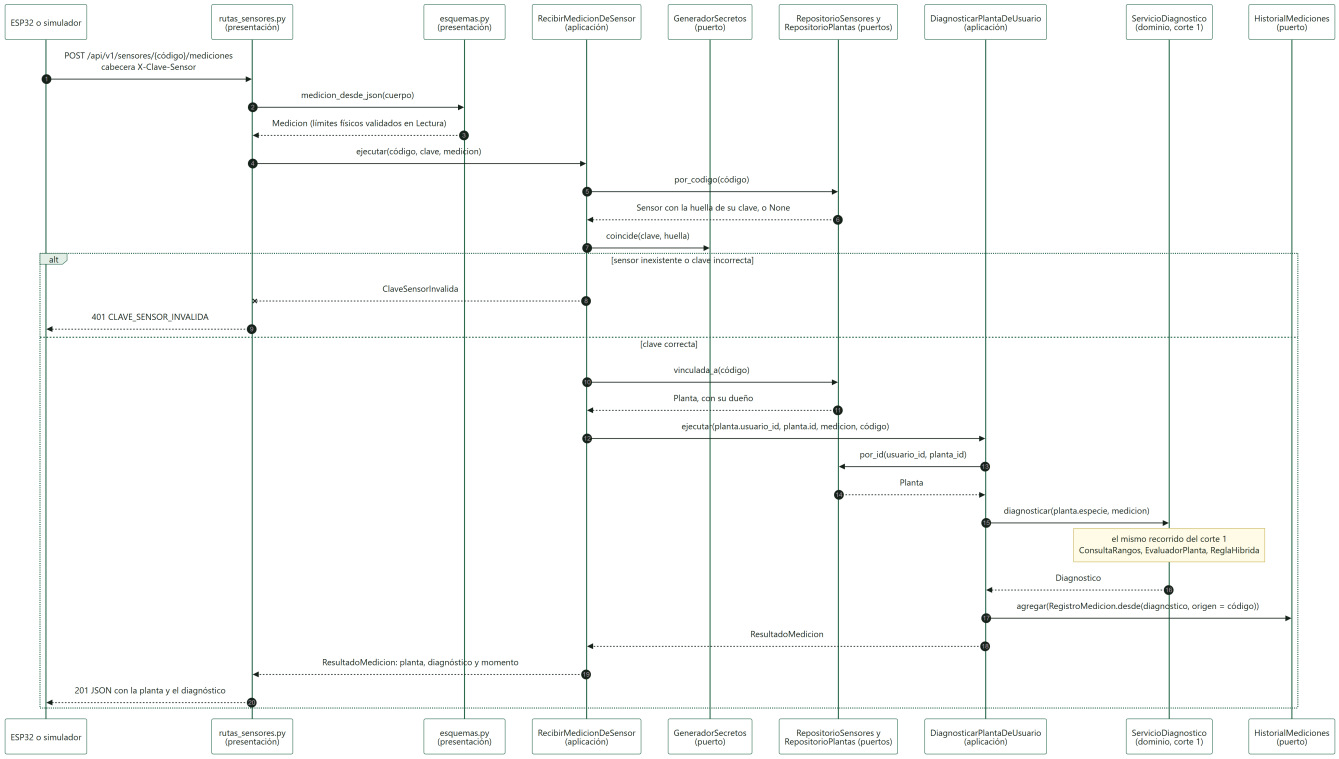


Figura 3. Medición de un sensor físico (fuente: secuencia-medicion-sensor.mmd).

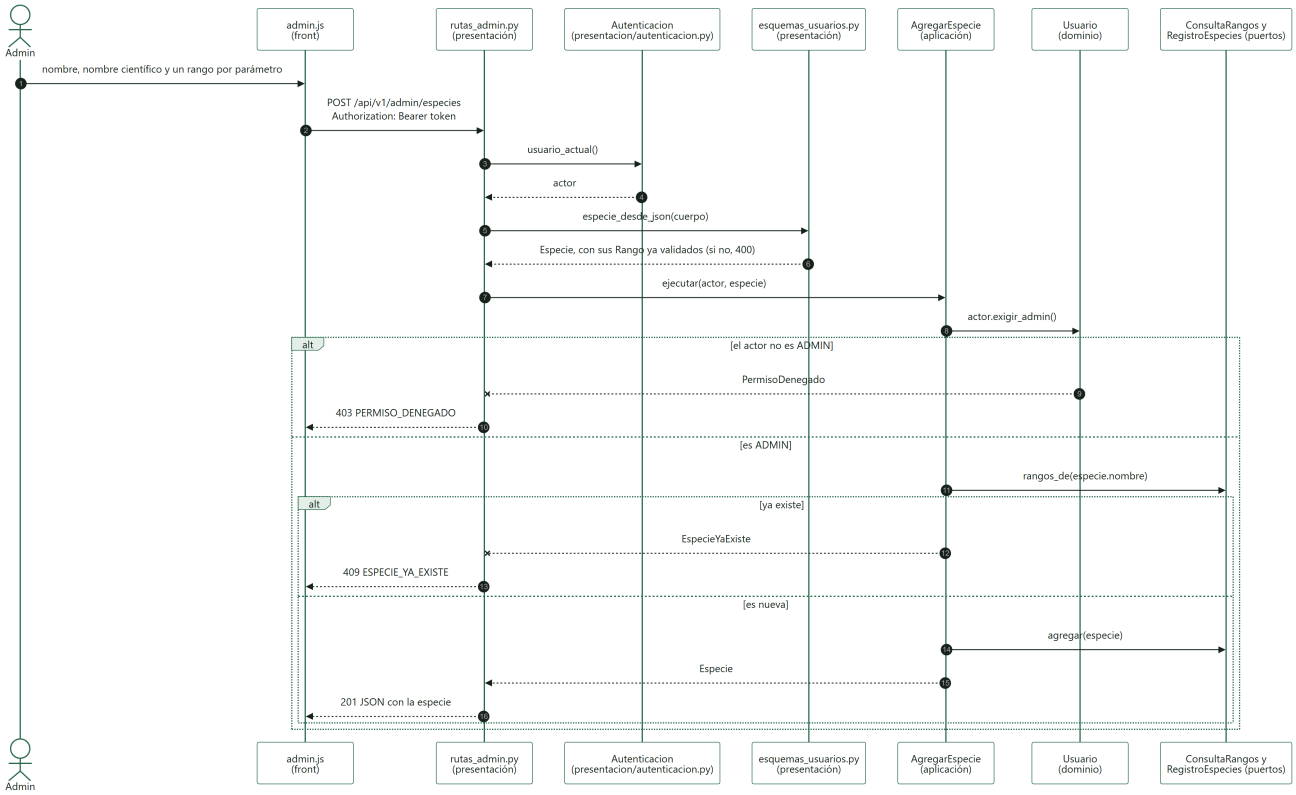


Figura 4. El admin agrega una especie (fuente: secuencia-admin-especie.mmd).

4. Responsabilidades por capa

Capa	Qué hace (lo nuevo del corte 2 en cursiva)	Depende de
Presentación presentacion/	rutas.py expone lo público (especies, parámetros, diagnóstico). *rutas_usuarios.py, rutas_admin.py, rutas_plantas.py y rutas_sensores.py traducen HTTP a los casos de uso nuevos. autenticacion.py saca el token de la cabecera Authorization y pregunta quién es; las rutas nunca leen cabeceras ni deciden permisos.*	Aplicación, dominio (objetos y errores), flask y werkzeug.
Aplicación aplicacion/	DiagnosticarPlanta, ListarEspecies, ListarParametros (corte 1). *cuentas.py (registro, login, logout, identificar), administracion.py (listar/activar/el rol de usuarios, agregar especies, cada caso empieza con actor.exigir_admin()), plantas.py, sensores.py, historial.py y diagnosticar_planta_de_usuario.py, que compone ServicioDiagnostico sin tocarlo.*	Dominio.
Dominio dominio/	Objetos y reglas del diagnóstico (corte 1), sin cambios. *usuarios.py (Usuario, Correo, Sesión, la regla exigir_admin()), plantas.py, sensores.py, historial.py y los puertos nuevos en puertos_usuarios.py: uno por agregado (RepositorioUsuarios, RepositorioSesiones, RepositorioPlantas, RepositorioSensores, HistorialMediciones), más Cifrador, GeneradorSecretos y Reloj.* Sigue sin importar flask, csv, http ni otra capa (lo comprueba tests/dominio/test_arquitectura.py).	Solo la biblioteca estándar de Python.
Infraestructura infraestructura/	*El CSV se eliminó. sqlite/base_datos.py prepara el esquema y precarga datos al arrancar; seis repositorios (RepositorioEspeciesSqlite y cinco más) implementan los puertos del dominio. seguridad.py implementa Cifrador (PBKDF2, 600 000 iteraciones) y GeneradorSecretos (SHA-256 sobre 32 bytes aleatorios) con hashlib/hmac/secrets. reloj.py implementa Reloj con datetime.now(utc).*	Dominio (puertos y objetos), sqlite3, flask y flask-cors.
Raíz de composición main.py	Crea las clases concretas y las conecta: ahora cinco blueprints en vez de uno, más cifrador, generador de secretos y reloj inyectados en cada caso de uso que los necesita.	Las cuatro capas.
Front frontend/	Diagnóstico y catálogo (corte 1). *Pantallas de acceso (login/registro), mis materas, vincular sensor, historial y panel de administración, todas pidiendo datos reales a la API; el token vive en sessionStorage.*	La API por HTTP (frontend/js/api.js).

5. Justificación SOLID (extendida al corte 2)

El corte 1 ya justificaba las cinco letras (ver sección archivada en docs/entrega/ del commit cf2e43d). Esta sección agrega, para cada letra, el ejemplo más claro que trajo el corte 2.

S — Responsabilidad única

Autenticacion (presentacion/autenticacion.py:13) solo hace dos cosas: sacar el token de la cabecera (token(), :17) y pedirle al caso de uso que lo resuelva (usuario_actual(), :24). No valida el token ella misma, no sabe qué es una huella SHA-256, no toca la base de datos. Esa lógica vive aparte, en **IdentificarUsuario.ejecutar()** (aplicacion/cuentas.py:113). Y **Usuario.exigir_admin()** (dominio/usuarios.py:77) es otro ejemplo: es la única función de todo el sistema que decide si alguien es administrador, y las rutas (rutas_admin.py) no repiten esa pregunta.

Sin aplicarlo: si Autenticacion también calculara huellas y consultara SQLite, cambiar el algoritmo de hash de los tokens obligaría a tocar la capa de presentación. Si cada ruta de admin repitiera «if usuario.rol != 'ADMIN'», bastaría con que alguien olvidara el chequeo en una ruta nueva para abrir un agujero de seguridad.

O — Abierto/cerrado

Cifrador y **GeneradorSecretos** (dominio/puertos_usuarios.py:112 y :122) son las abstracciones. IniciarSesion y RegistrarUsuario dependen de Cifrador, nunca de CifradorPbkdf2 (infraestructura/seguridad.py:16) directamente. Cambiar el algoritmo de hash de contraseñas —de PBKDF2 a bcrypt o argon2, por ejemplo— es escribir una clase nueva que implemente Cifrador y cambiar una línea en main.py; ningún caso de uso de cuentas se toca. Lo mismo para cambiar de SQLite a otra base: los seis repositorios se reemplazan uno por uno sin que aplicacion/ ni dominio/ se enteren.

Sin aplicarlo: si IniciarSesion llamara a hashlib.pbkdf2_hmac() por su cuenta, cambiar de algoritmo tocaría la lógica de login, y sus pruebas (que hoy usan un doble de Cifrador que no hace ningún cálculo real) tendrían que esperar a que termine un PBKDF2 real de 600 000 iteraciones.

L — Sustitución de Liskov

RepositorioPlantas.por_id() (dominio/puertos_usuarios.py:73) tiene un contrato explícito en su docstring: «None si la planta no existe o si es de otro usuario». Tanto RepositorioPlantasSqlite.por_id() (infraestructura/sqlite/repositorio_plantas.py:50, un WHERE id = ? AND usuario_id = ?) como el doble en memoria de las pruebas cumplen exactamente eso: ninguna implementación devuelve una planta ajena, y buscar_planta_propia() (aplicacion/plantas.py) confia en esa garantía sin preguntar qué clase concreta recibió.

Sin aplicarlo: si una implementación futura (por ejemplo, para pruebas de carga) devolviera la planta sin filtrar por dueño «porque total nadie más la pide», cualquier caso de uso que confiara en el contrato quedaría exponiendo plantas ajenas sin que ninguna prueba de dominio lo detectara, porque las pruebas de dominio no conocen la implementación SQL.

I — Segregación de interfaces

El corte 2 repite la decisión del corte 1 (puertos de un solo propósito) a mayor escala: en vez de un RepositorioUsuarios gigante con todo adentro, hay cinco puertos separados por agregado (RepositorioUsuarios, RepositorioSesiones, RepositorioPlantas, RepositorioSensores, HistorialMediciones) más RegistroEspecies aparte. DiagnosticarPlanta (el caso de uso público del corte 1) sigue dependiendo solo de ConsultaRangos: no sabe que ahora existen usuarios ni sesiones, aunque compartan la misma base SQLite por dentro.

Sin aplicarlo: un único RepositorioUsuarios con métodos de plantas, sensores e historial mezclados obligaría a cualquier doble de prueba —incluso uno que solo necesita por_correo()— a implementar métodos que nunca usa, como se explicó en el corte 1 para ConsultaRangos y CatalogoEspecies.

D — Inversión de dependencias

RepositorioUsuariosSqlite.guardar() (infraestructura/sqlite/repositorio_usuarios.py:22) implementa el puerto RepositorioUsuarios; la dependencia va de infraestructura hacia el dominio, como en el corte 1. main.py sigue siendo el único lugar que elige las clases concretas: construye BaseDatos, los seis repositorios, CifradorPbkdf2, GeneradorSecretosSeguro y RelojSistema, y los inyecta en cada caso de uso por constructor.

Tensión (igual que en el corte 1): DiagnosticarPlantaDeUsuario depende de la clase concreta ServicioDiagnostico, no de una abstracción —se aceptó por la misma razón: ambas capas son internas y estables, y es justamente la conexión deliberada entre el núcleo del corte 1 y el corte 2, documentada como tal en el plan de evolución original (sección 5.3 del corte 1).

6. Modelo de datos

backend/datos/esquema.sql define siete tablas. Tres conservan el patrón del corte 1 (especie y rango_optimo: una fila por parámetro, para que agregar el pH no cambie el esquema). Las otras cuatro son del corte 2.

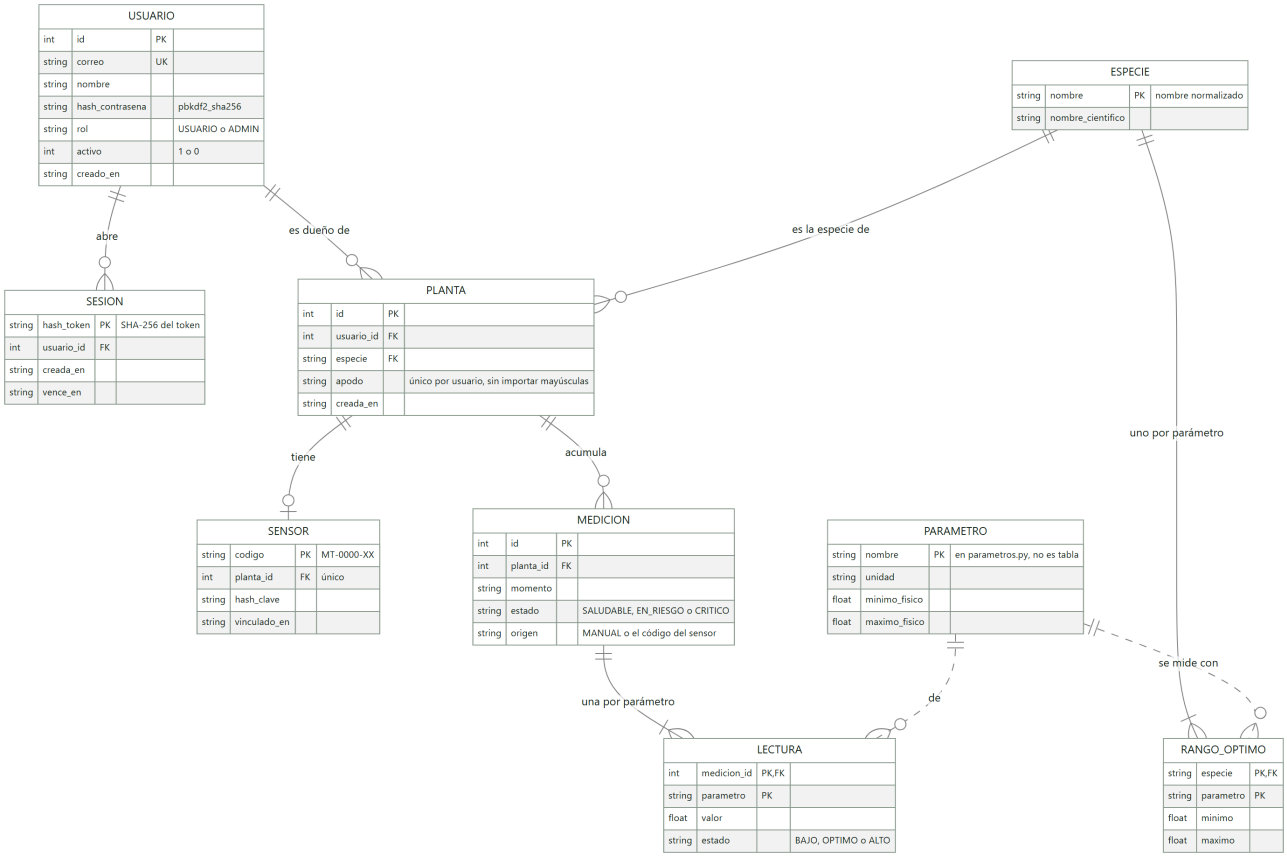


Figura 5. Entidad-relación del corte 2 (fuente: entidad-relacion-corte2.mmd).

El patrón de seguridad se repite tres veces en el esquema: usuario.hash_contrasena, sesion.hash_token y sensor.hash_clave guardan siempre un hash (PBKDF2 o SHA-256, según el caso), nunca el secreto real. Los borrados en cascada (ON DELETE CASCADE) hacen que eliminar un usuario o una planta limpie automáticamente sus sesiones, plantas, sensores e historial dependientes, sin que ningún caso de uso tenga que acordarse de borrar tabla por tabla.

6.1 Seguridad de credenciales y sesiones

Secreto	Cómo se protege	Dónde
Contraseña de usuario	PBKDF2-HMAC-SHA256, 600 000 iteraciones, sal aleatoria de 16 bytes por cuenta	usuario.hash_contrasena
Token de sesión	SHA-256 de 32 bytes aleatorios (no hace falta una función lenta: no hay diccionario que probar)	sesion.hash_token
Clave de un sensor	SHA-256, mismo motivo que el token de sesión	sensor.hash_clave

No es JWT: la sesión es un token opaco. La validez depende de que exista una fila en la tabla sesion (IdentificarUsuario.ejecutar(), aplicacion/cuentas.py:113), no de una firma criptográfica autocontenida. Esto permite revocar una sesión al instante —al cerrar sesión o al desactivar una cuenta (CambiarEstadoUsuario borra todas sus sesiones, aplicacion/administracion.py)—, algo que un JWT firmado no permitiría sin mantener una lista de revocación aparte, que terminaría siendo el mismo problema con otro nombre.

7. Diagramas de clases

El detalle completo, capa por capa, vive en docs/diagramas/ (clases-dominio.mmd, clases-puertos.mmd, clases-aplicacion.mmd y clases-cuentas.mmd), con borde continuo para lo del corte 1 y punteado para lo del corte 2 en los primeros tres. Acá se incluye el de cuentas y administración, por ser el más nuevo.

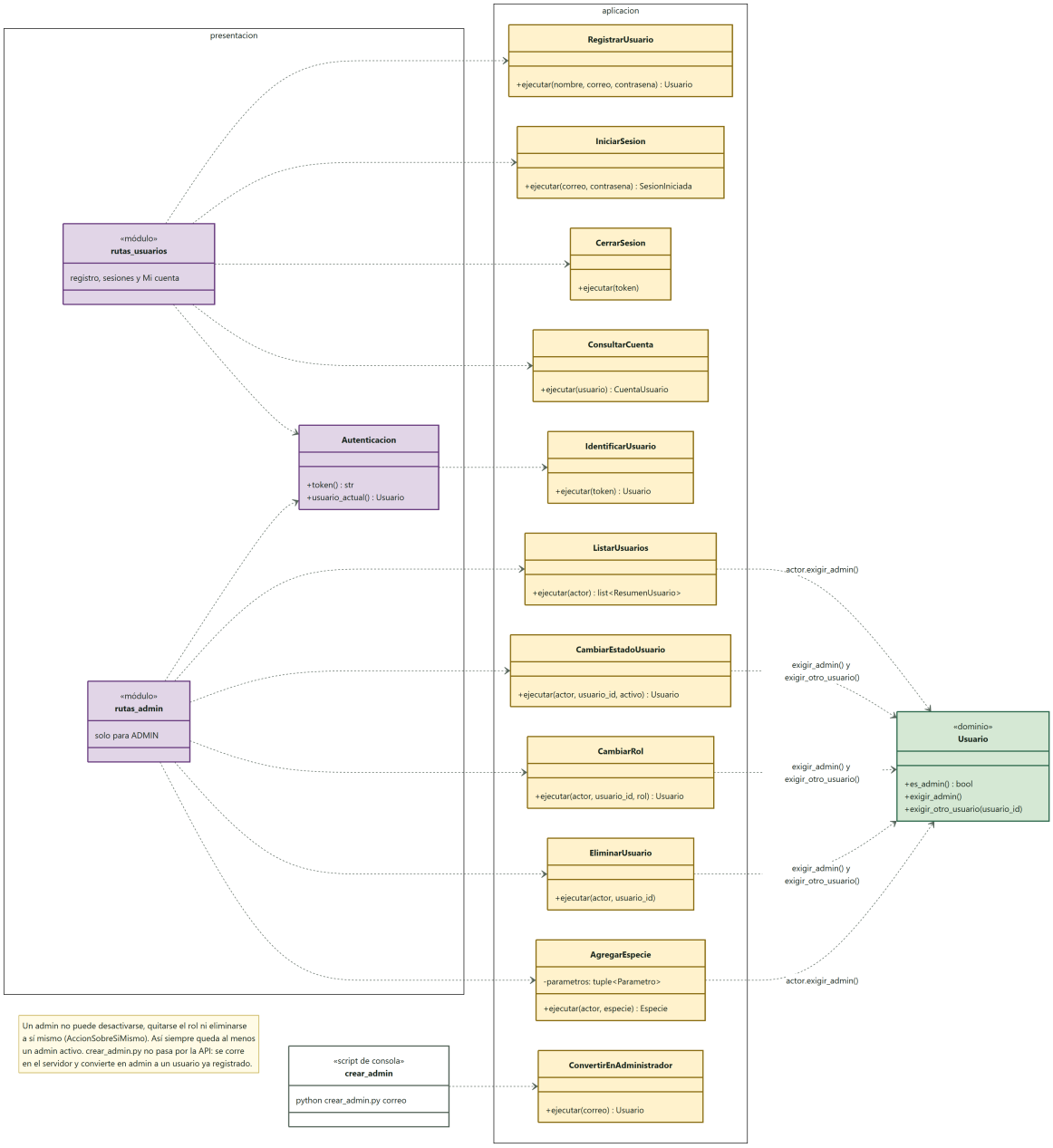


Figura 6. Presentación y aplicación de cuentas y administración (fuente: clases-cuentas.mmd).

8. Plan de evolución (actualizado)

Dos de los cuatro escenarios que planteaba el corte 1 ya están implementados: la base de datos relacional (era la sección 5.2) y los usuarios con sus plantas (era la 5.3). Quedan dos pendientes.

8.1 Las mediciones llegan por MQTT desde un ESP32

Se agrega infraestructura/mqtt_listener.py: conexión al broker y suscripción al tópico de mediciones (paho-mqtt). El mensaje se traduce a una Medicion igual que hace hoy rutas_sensores.py con el JSON del ESP32 real; los límites físicos se validan solos, en el dominio.
Se modifica main.py (arranca el listener) y requirements.txt.
No se toca dominio/, aplicación/ ni RecibirMedicionDeSensor: MQTT sería otra entrada al mismo caso de uso que ya usa la ruta REST del sensor, exactamente como se diseñó desde el corte 1. Como referencia, MATERA_HOST (infraestructura/configuracion.py, agregado en el commit fff4ebb) ya deja la API escuchando en la red local, un paso previo necesario para que el ESP32 real le hable al servidor por HTTP mientras no exista el listener MQTT.

8.2 Tiempo real: WebSocket para «mis materas»

Se agrega presentacion/websocket.py: al_conectar(data) identifica al usuario con el mismo Autenticacion de hoy, y al_recibir_medicion(data) reutiliza DiagnosticarPlantaDeUsuario, el mismo caso de uso de la ruta REST. Un NotificadorWebSocket implementaría un puerto nuevo, NotificadorDiagnosticos, para empujar el diagnóstico al navegador sin que el usuario refresque la página.
Se modifica main.py (conecta el notificador) y el front (escucha el socket en vez de solo pedir datos).
No se toca ningún caso de uso existente: es exactamente la misma idea que MQTT, otra entrada al mismo núcleo. El costo real no es de diseño sino operativo: conexiones persistentes, reconexión, y autenticar un socket (no hay cabecera Authorization en una conexión ya abierta).

8.3 Se agrega gamificación: puntos, rachas y niveles

Sin cambios respecto al plan del corte 1 (sección 5.4 original): módulo de dominio propio gamificacion.py, con RegistrarProgreso envolviendo a DiagnosticarPlantaDeUsuario (ahora el del corte 2, no ya DiagnosticarPlanta) con un decorador, sin modificarlo. Requiere el corte 2 completo, porque los puntos son de un usuario.

9. Decisiones y alternativas descartadas (corte 2)

Decisión	Alternativa descartada	Por qué se descartó
Token opaco de sesión: se guarda el SHA-256 del token, nunca el token (dominio/usuarios.py, clase Sesión).	JWT firmado.	Cerrar sesión o desactivar una cuenta no invalida un JWT ya emitido sin mantener una lista de revocación aparte, que sería el mismo problema con otro nombre. El token opaco se revoca borrando una fila.
SQLite con una conexión por operación (BaseDatos.conexion()).	Una conexión compartida para todo el servidor.	Flask atiende cada petición en un hilo distinto, y un objeto sqlite3.Connection no se puede usar desde otro hilo; una conexión compartida fallaría en el servidor real aunque las pruebas con test_client pasaran.
Planta ajena responde 404 (PlantaNoEncontrada), igual que una inexistente.	403 (acceso prohibido) cuando la planta existe pero es de otro usuario.	Un 403 confirmaría que esa planta existe, filtrando información sobre las plantas de otra cuenta.
Cada repositorio nuevo es un puerto de un solo agregado (RepositorioUsuarios, RepositorioPlantas, etc.).	Un único repositorio grande para todo el corte 2.	Mismo motivo que ConsultaRangos/CatalogoEspecies en el corte 1 (ISP): cada cliente depende solo de lo que usa, y los dobles de prueba quedan mínimos.
La regla de permisos vive en el dominio (Usuario.exigir_admin()), no en las rutas HTTP.	Revisar el rol en cada ruta de administración (if usuario.rol != 'ADMIN').	La regla se cumple igual sin importar el canal de entrada (HTTP, consola, un futuro MQTT o WebSocket); repetirla en cada ruta arriesgaba que alguien la olvidara en una ruta nueva.
Un admin no puede desactivarse, quitarse el rol ni eliminarse a sí mismo (exigir_otro_usuario()).	Permitirlo y confiar en que un admin no se desactive por error.	Sin esta regla, el sistema podría quedarse sin ningún admin activo; no existe ninguna ruta pública para volver a dar el rol, solo el script de consola.